A Novel Approach For On-line Testable Reversible Logic Circuit Design

D. P. Vasudevan, P. K. Lala and J. P. Parkerson
Department of Computer Science and Computer Engineering
University of Arkansas, Fayetteville, AR 72701, USA
E-mail: lala@engr.uark.edu

Abstract

Two testable reversible logic gates are proposed in this paper. These gates can be used to implement reversible digital circuits with various levels of complexity. The major feature of these gates is that they provide online-testability for circuits implemented using these gates. The application of these gates in testable ripple carry, carry–skip adders and MCNC benchmark circuits have been illustrated.

1. Introduction

Reversible Logic circuits are of major interest in low power CMOS design, Quantum Computing and Nanotechnology based systems. A computational operation is said to be logically reversible if the logical state of the computational device before the operation of the device can be determined by its state after the operation i.e., the input of the system can be retrieved from the output obtained from it. Irreversible erasure of a bit in a system leads to generation of energy in the form of heat. An operation is said to be physically reversible if it converts no energy to heat and produces no entropy. Landauer has shown that for every bit of information lost in logic computations that are not reversible, kTlog2 joules of heat energy is generated, where k is Boltzmann's constant and the absolute temperature at which computation is performed [1]. The amount of energy dissipation in a system increases in direct proportion to the number of bits that are erased during computation. Bennett showed that kTln2 energy dissipation would not occur, if a computation were carried out in a reversible way [2]. Reversible computation in a system can be performed if the system is composed of reversible gates.

2. Reversible Logic Gate

A gate is reversible, if there is a distinct output assignment for each distinct input. Thus, a reversible gate's inputs can be uniquely determined from its outputs. A reversible logic gate must have the same number of inputs and outputs [2]. In an n-output reversible gate the output vectors are a permutation of the numbers 0 to 2^n-1. Reversible gates are balanced, i.e. the outputs are 1s for exactly half of the inputs. A circuit without constants on its inputs and composed of reversible gates realizes only balanced functions. It can realize non-balanced function only with garbage outputs. Some of the major problems with reversible logic synthesis are [3]:

i) Fan-outs are not allowed
ii) Feedback from gate outputs to inputs are not permitted

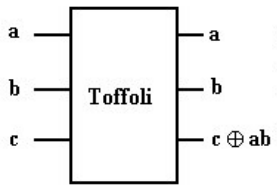
A logic synthesis technique using reversible gate should have the following features [3]:

i) Use minimum number of garbage outputs
ii) Use minimum input constants
iii) Keep the length of cascading gates minimum
iv) Use minimum number of gates

Several reversible gates have been proposed over the years, e.g., the Toffoli gate, the Fredkin gate etc., [4]. In Toffoli Gate, all the inputs from 1 to (n-1) are passed as outputs. The nth output is controlled by 1 to (n-1) inputs. When all the inputs from 1 to (n-1) are 1s, the nth input is inverted and passed as output else original signal is passed. A 3-input, 3-output Toffoli gate is shown in Figure 1. The inputs 'a' and 'b' are passed as first and second output respectively. The third output is controlled by 'a' and 'b' to invert 'c'. The Fredkin gate is shown in the Figure 2. Here the input 'a' is passed as first output. Inputs b and c are swapped to get the second and third output which is controlled by 'a'. Thus two inputs can be swapped by controlling the swap using another input in

Proceedings of the 13th Asian Test Symposium (ATS 2004)
0-7695-2235-1/04 $20.00 © 2004 IEEE

Fredkin Gate. Truth tables for 3-input and 3-output Toffoli Gate and Fredkin gates are shown in Table 1 and Table 2 respectively.



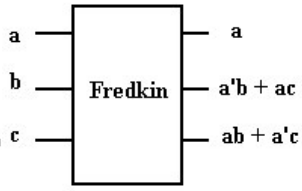


Figure 1: Toffoli Gate Figure 2: Fredkin Gate

3. Testable Reversible Gates

The currently available reversible gates can be used to implement arbitrary logic functions, however the testing of such circuits as far as we aware of, has not been addressed in literature.

Table 1: Truth Table – Toffoli Gate

A	B	C		X	Y	Z
0	0	0		0	0	0
0	0	1		0	0	1
0	1	0		0	1	0
0	1	1		0	1	1
1	0	0		1	0	0
1	0	1		1	0	1
1	1	0		1	1	1
1	1	1		1	1	0

Table 2: Truth Table – Fredkin Gate

A	B	C		X	Y	Z
0	0	0		0	0	0
0	0	1		0	1	0
0	1	0		0	0	1
0	1	1		0	1	1
1	0	0		1	0	0
1	0	1		1	0	1
1	1	0		1	1	0
1	1	1		1	1	1

The testing of reversible logic gates can be a problem because the levels of logic can be significantly higher than in standard logic circuits. This paper proposes two reversible logic gates, R1 and R2 that can be used in pairs to design testable reversible logic circuits. The first gate R1 is used for implementing arbitrary functions while the second gate R2 is employed to incorporate on-line testability features into the circuit. Gates R1 and R2 are shown in Figure 3 a 1 and Figure 3 a 2 respectively; the corresponding truth tables of the gates are shown in the Table 3 and Table 4 respectively. It can be verified from the truth tables that the input pattern corresponding to a particular output pattern can be uniquely determined.

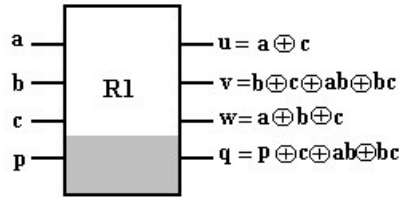


Figure 3. a. 1: New Reversible Logic Gate 1

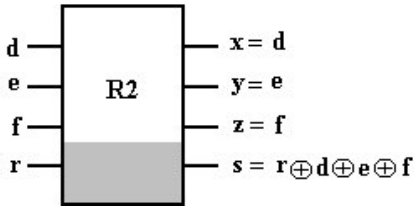


Figure 3. a. 2: New Reversible Logic Gate 2

Table 3:
Truth Table - Proposed Reversible Logic Gate1

p	a	b	c		u	v	w	q
0	0	0	0		0	0	0	0
0	0	0	1		1	1	1	1
0	0	1	0		0	1	1	0
0	0	1	1		1	1	0	0
0	1	0	0		1	0	1	0
0	1	0	1		0	1	0	1
0	1	1	0		1	0	0	1
0	1	1	1		0	0	1	1
1	0	0	0		0	0	0	1
1	0	0	1		1	1	1	0
1	0	1	0		0	1	1	1
1	0	1	1		1	1	0	1
1	1	0	0		1	0	1	1
1	1	0	1		0	1	0	0
1	1	1	0		1	0	0	0
1	1	1	1		0	0	1	0

Proceedings of the 13th Asian Test Symposium (ATS 2004)
0-7695-2235-1/04 $20.00 © 2004 **IEEE**

Gate R1 can implement all Boolean functions; during normal operation input p is set to 0. The OR and the EX-OR functions can be simultaneously implemented on R1 (Figure 3 b). The EX-NOR function and the NAND function are obtained by setting input c to 1 (Figure 3 c). The NOR function can be obtained by cascading two R1 gates (Figure 3 d). An AND gate also requires cascading of two gates (Figure 3 e). R1 can transfer a signal at input a to output x by setting input c to 0.

4:Truth Table- Proposed Reversible Logic Gate2

r	d	e	f		x	y	z	s
0	0	0	0		0	0	0	0
0	0	0	1		0	0	1	1
0	0	1	0		0	1	0	1
0	0	1	1		0	1	1	0
0	1	0	0		1	0	0	1
0	1	0	1		1	0	1	0
0	1	1	0		1	1	0	0
0	1	1	1		1	1	1	1
1	0	0	0		0	0	0	1
1	0	0	1		0	0	1	0
1	0	1	0		0	1	0	0
1	0	1	1		0	1	1	1
1	1	0	0		1	0	0	0
1	1	0	1		1	0	1	1
1	1	1	0		1	1	0	1
1	1	1	1		1	1	1	0

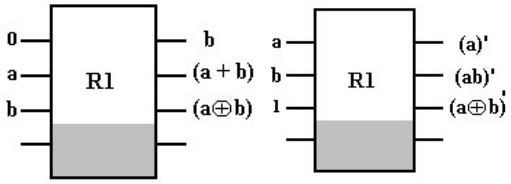


Figure 3 b: OR and EXOR Gates
Figure 3 c: XNOR and NAND

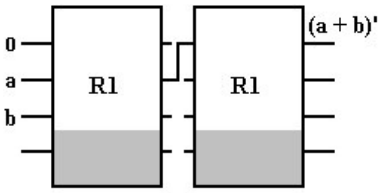


Figure 3 d: NOR Gate

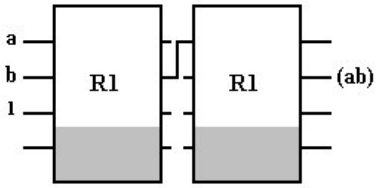


Figure 3 e: AND Gate

Gate R2 is used to transfer the input values at d, e, f to outputs x, y, z; it also generates the parity of the input pattern at output s. The output s of the gate is the complement of the input r if all other inputs of the gate remain unchanged. For example, if input d e f r = 1000 is changed to 1001, the output of the gate will change from 1000 to 1001. During normal operation input r is set to 1. A testable logic block can be formed by cascading R1 and R2 as shown in Figure 4. In this configuration gate R2 is used to check on-line whether there is a fault in R1 or in itself. If R1 is fault free, its parity output q and the parity output s of R2 should be complementary, otherwise the presence of a fault is assumed. Thus during normal operation the presence of a fault in the logic block can be detected.

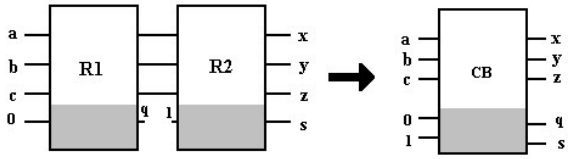


Figure 4: Testable Block

4. Application of the Testable Block

To illustrate the application of the logic block of Figure 4 two different types of adders- ripple carry and carry skip adders and two MCNC benchmark circuits are designed using this block.

4.1 Ripple Carry Adders

The basic building block of a ripple carry adder is a full adder circuit. A full adder computes the sum bit S_i and carry output bit C_{i+1} based on addend input bits a and b and carry input c. The output expressions for a Ripple Carry adder are:

$S_i = a \oplus b \oplus c;$

$C_{i+1} = ab + bc + ca; \quad (i = 0,1,2,\ldots.)$

The sum output S_i is obtained at the output of the third gate (from the left) and the carry output C_{i+1} is obtained at the fourth gate output. Figure 5 shows the implementation of the ripple carry adder. The sum output and the carry output of the adder are obtained as shown in the Figure 5. The q and s outputs of each block adder are fed into a two-rail checker that generates 01 or 10 outputs in the absence of a fault; otherwise the checker produces 00 or 11 outputs.

4.2 Carry Skip Adders

Carry skip adders reduce the carry computation delay. A full adder with a propagate signal is used as a building block in carry skip adders. In a full adder, if either input is a '1',the carry input is propagated to the carry output. Therefore, the carry input C_i to the ith full adder will propagate to its carry output C_{i+1} when $p = a \oplus b = 1$; such an adder is called a carry skip compatible adder [5].

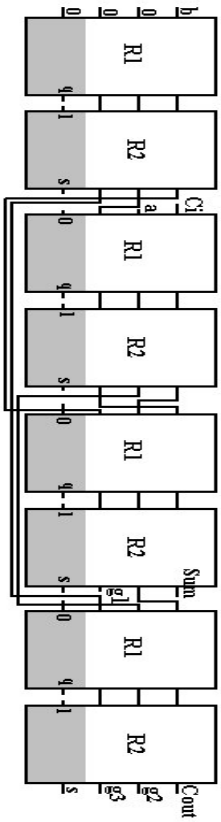


Figure 5: Full adder without propagate using proposed testable block

The carry skip compatible adder designed using the proposed testable blocks is shown in the Figure 6. Each gate output can be checked for faults using pin q and pin s of the corresponding gate. As said earlier, the value of pins q and s indicate the information about a fault on that particular block. A 4-bit carry skip adder can be constructed by using four carry-propagate compatible full adders as shown in Figure 7 a. The carry skip adder consists of four carry-propagate compatible full adders. The propagate signals P0, P1, P2 and P3 generated by each adder are ANDed. The resulting output is ANDed with carry input Ci. The corresponding output is ORed with the carry output of the fourth full adder to get the carry output as shown in Figure 7 a. Four NFA (New Full Adder) blocks used in the design are carry skip compatible full adder blocks designed using the proposed reversible gate shown in Figure 7 b. The propagate signals are ANDed and the carry output of the fourth NFA.

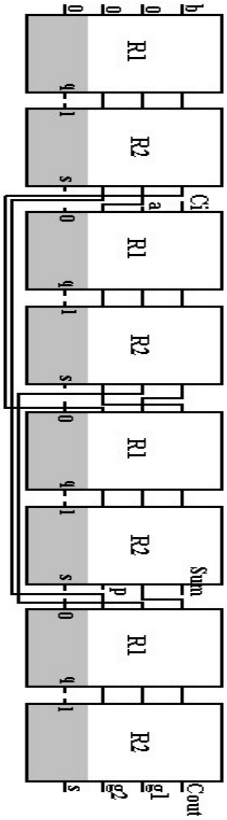


Figure 6: Full adder with propagate using proposed gate

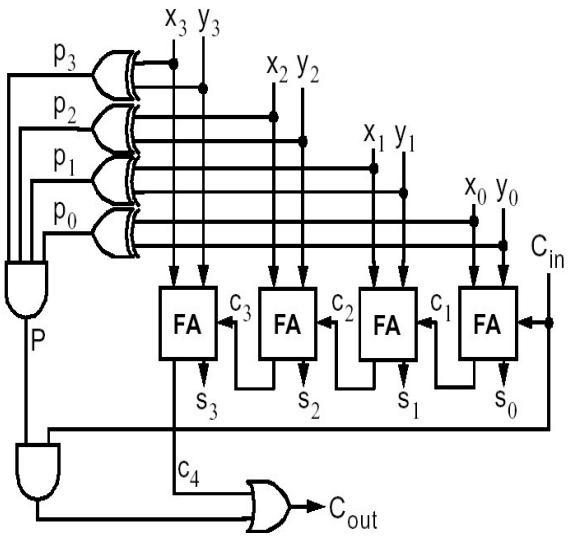


Figure 7 a: Architecture of 4-bit Carry skip adder

The carry skip adder constructed using the two proposed gates with testable features is shown in the Figure 7 b. Connecting the q and s outputs of the testable blocks to a two-rail checker can test the carry skip adder circuit.

4.3 MCNC Benchmarks

Two MCNC benchmark circuits named Function CLIP and Xor5 have been implemented using the proposed gate.

The function Xor5 is given by

{xor5} = [1] [2]' + [1]' [2]
[1] = [3] e' + [3]' e
[2] = a b' + a' b
[3] = c d' + c' d

This function has been implemented by using only four testable blocks. Three checker blocks will be needed to implement the online testability. Figure 9 shows the implementation of the CLIP function. The function is given by

f = e' [4] + e [4] + [2] . [3] + a . d + b . c
[2] = b + c ; [3] = a + d; [4] = [2] + [3]

For implementing this function ten testable blocks composed of the proposed reversible gates are employed. The pins q and s of each gate are connected to two rail checkers; thereby the online testability feature is obtained. It needs 9 checker blocks to realize the online testing functionality.

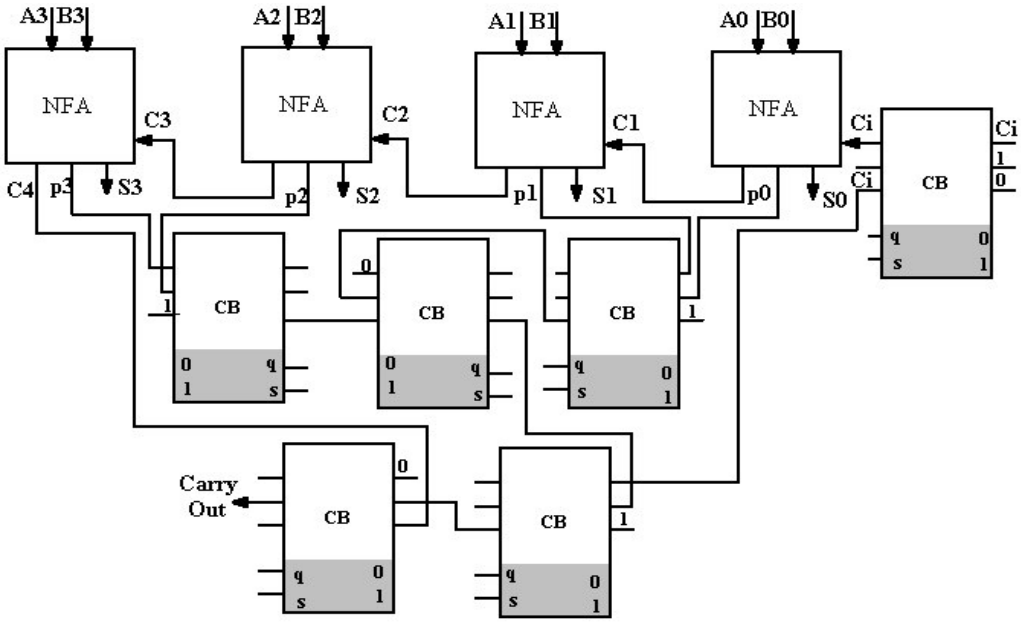


Figure 7b: Four bit carry skip adder using proposed gate

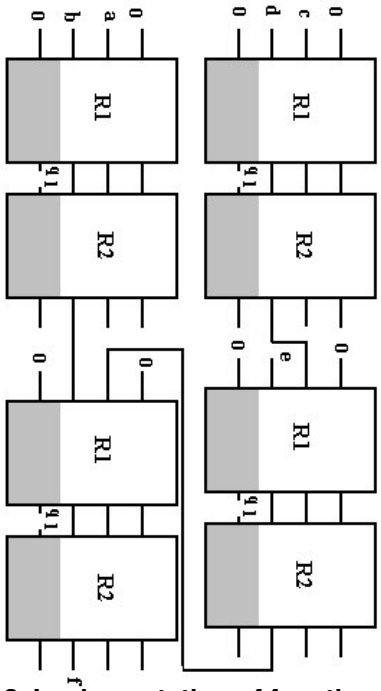


Figure. 8. Implementation of function Xor5 using Proposed Gates

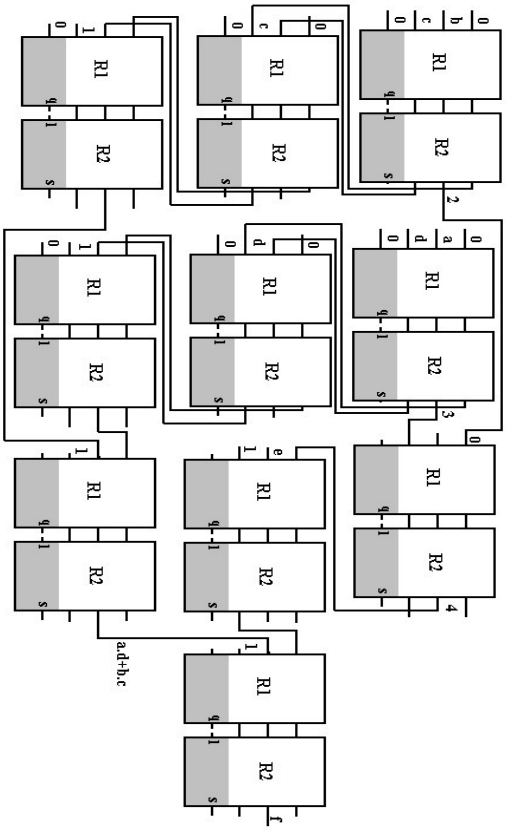


Figure. 9. Implementation of function CLIP using Proposed Gates

5. Conclusion

Two testable reversible logic gates have been proposed. By using the two gates as a pair, any arbitrary function can be implemented. The resulting circuits can be tested on-line. In addition a fault in any of these gates that produces a single bit output error can be detected by the built-in parity. Work is currently in progress for developing a two-rail checker using these gates in order to realize fully on-line testable reversible logic designs.

References

[1] R. Landauer, "Irreversibility and Heat Generation in the Computing Process", IBM J. Research and Development, vol. 3, pp. 183-191, July 1961.

[2] C. H. Bennett, "Logical Reversibility of Computation", IBM J. Research and Development, pp. 525-532, November 1973.

[3] M Perkowski and P Kerntopf, "Reversible Logic", invited tutorial, Proc, EURO-MICRO, Warsaw, Poland, Sept 2001.

[4] E. Fredkin and T. Toffoli, " Conservative Logic", Intl. J. Theoretical Physics, vol. 21, nos.3-4, pp. 219-253, 1982.

[5] T.Sasao and K. Kinoshita, " Conservative Logic Elements and Their Universality", IEEE Trans. Computers, vol. 28, no. 9, pp. 682-685, 1979.